



Algoritmos e Programação

Prof. Me. Fiorin
andre.fiorin@iffarroupilha.edu.br



Resumindo

- Na última aula falamos sobre...
 - Estruturas de dados homogêneas
 - Vetores
 - Matrizes



Aula 09

Modularização



Modularização

- Na medida em que os problemas se tornam mais complexos e extensos, percebemos que a melhor saída é dividi-lo em pequenos problemas.
 - Códigos repetidos
 - Códigos muito extensos que dificultam a compreensão
 - ...
- Solução: **Dividir para conquistar!**



Modularização

- Vantagens

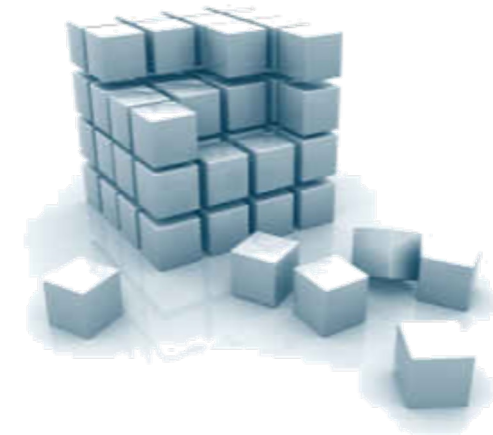
- Facilita a detecção de erros
- Facilita a tarefa de testes, afinal é mais fácil testar os módulos individualmente do que o programa completo.
- Facilita a manutenção
 - Correção de erros
 - Melhoramentos
- Permite o desenvolvimento independente dos módulos, simplificando e reduzindo o problema.
- Reutilização do código desenvolvido



Modularização

- Rotinas e sub-rotinas

- Sabemos que um programa é uma rotina (algoritmo) com muitas instruções.
- A modularização é a criação de sub-rotinas
 - É um bloco de código bem definido, com a finalidade de resolver uma única tarefa.
 - Podem ser chamados pelo programa (rotina principal/main) ou por outras sub-rotinas.





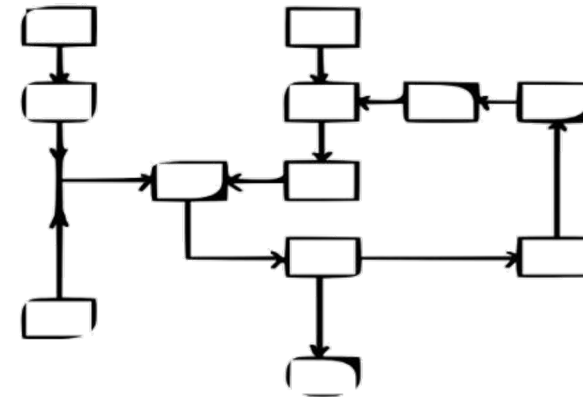
Modularização

- As sub-rotinas podem ser divididas em dois grupos
 - Procedimentos
 - Desviam o fluxo de execução do programa, sem retornar valores para a rotina chamadora.
 - Funções
 - Retornam valores para a rotina chamadora.



Procedimentos

Sub-rotinas sem valor de retorno





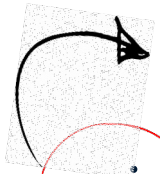
Procedimentos

- São as sub-rotinas mais comuns e mais fáceis de implementar.
- Desviam o fluxo de execução para a primeira instrução do procedimento.
- Ao final da execução do procedimento, o fluxo de execução retorna para o código que o chamou, podendo ser o programa principal (main) ou outra sub-rotina.



Procedimentos

- Sintaxe



O tipo de retorno dos procedimentos sempre deve ser "void"

```
void nome_procedimento (<lista de parâmetros>){  
    <lista de variáveis>  
    <lista de constantes>  
    <bloco de comandos/instruções>  
}
```

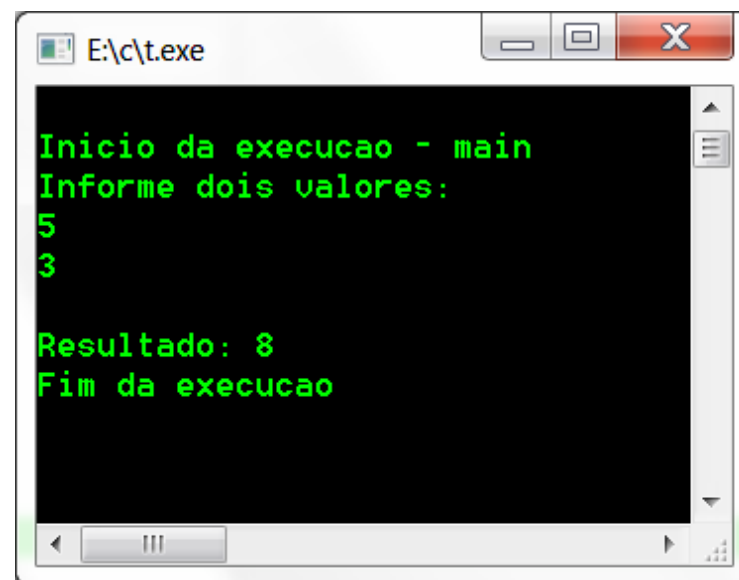
Procedimentos

- Exemplo

```

1 #include<stdio.h>
2 // procedimento
3 void soma () {
4     int a, b, result;
5     printf("\nInforme dois valores: \n");
6     scanf("%d", &a);
7     scanf("%d", &b);
8     result = a + b;
9     printf("\nResultado: %d", result);
10 }
11
12 // rotina principal
13 main (void) {
14     printf("\nInicio da execucao - main");
15
16     soma();           // invocando o procedimento
17
18     printf("\nFim da execucao");
19     getch();
20 }

```





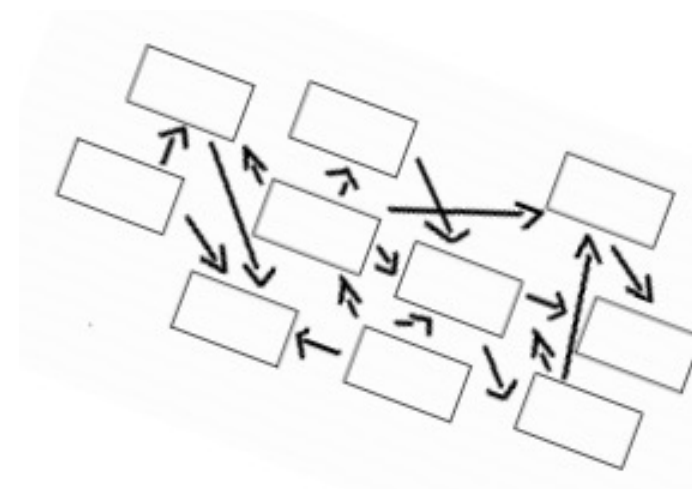
Exercício 01

- Faça um programa que contenha uma estrutura de caso que apresente as 4 operações matemáticas básicas. Cada opção deve invocar um procedimento para realizar a respectivo operação baseado em valores informados pelo usuário.



Funções

Sub-rotinas com valor de retorno



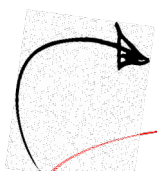


Funções

- São tipos especiais de sub-rotinas que além de desviar o fluxo de execução normal (de forma análoga aos procedimentos), retornam **um** valor.
- **Sempre possui um tipo de retorno.**
- O retorno pode ser uma expressão, uma constante ou uma variável.

Funções

- Sintaxe



O tipo de retorno das funções não pode ser "void"

```
tipo_retorno nome_função (<lista de parâmetros>){  
    <lista de variáveis>  
    <lista de constantes>  
    <bloco de comandos/instruções>  
    return <valor>;  
}
```

Funções

Exemplo

```

1 #include<stdio.h>
2
3 int verifica(){
4     int a, b;
5     printf("\nInforme dois valores (funcao): \n");
6     scanf("%d", &a);
7     scanf("%d", &b);
8     if (a != b)
9         return 0;
10    else
11        return 1;
12 }
13
14 main (void){
15     int result;
16     printf("\nInicio da execucao (main)");
17
18     result = verifica();
19
20     if (result)
21         printf("\n Os valores informados sao iguais (main)");
22     else
23         printf("\n Os valores informados sao diferentes (main)");
24     getch();
25 }

```

```

E:\c\t.exe

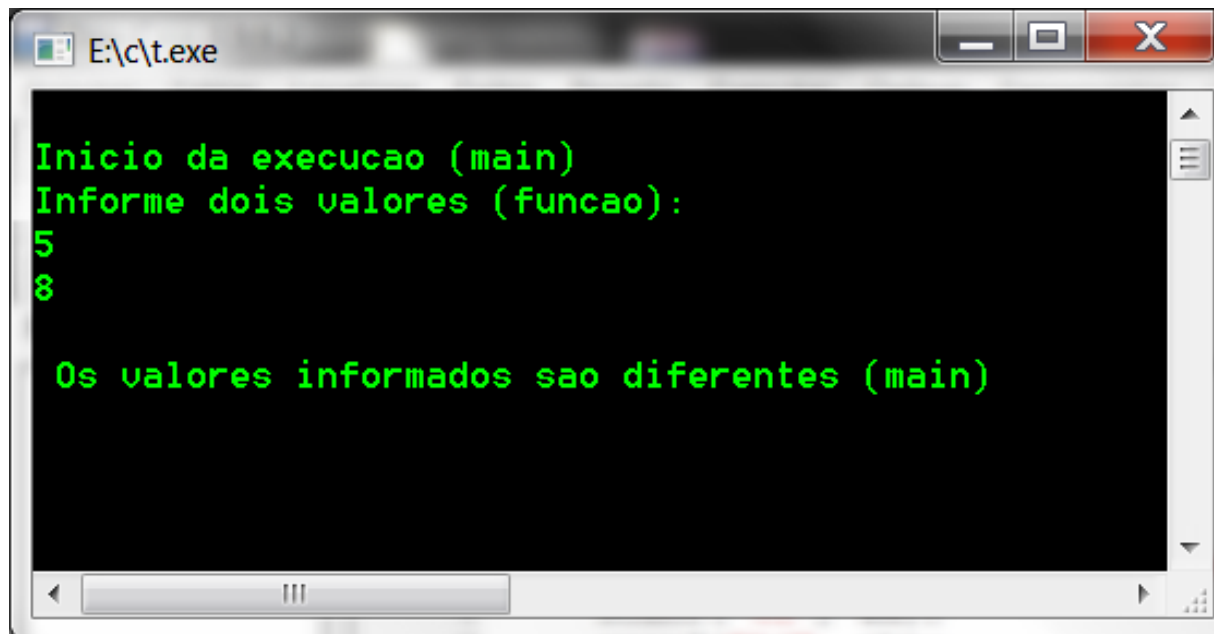
Inicio da execucao (main)
Informe dois valores (funcao):
5
8

Os valores informados sao diferentes (main)

```


Funções

- Exemplo



```

E:\c\t.exe

Inicio da execucao (main)
Informe dois valores (funcao):
5
8

Os valores informados sao diferentes (main)
  
```



Exercício 02

- Escreva um programa que contenha uma função para calcular e retornar a soma dos valores de 0 até N, onde N é o N-ésimo elemento da sequência de Fibonacci. A sequência e o resultado da soma devem ser apresentados na tela.

Passagem de Parâmetros

parâmetros

/ \

```
function add(x, y) {  
    return x + y;  
}
```

```
sum = add(4, 1);
```

| /

argumentos



Passagem de Parâmetros

- A passagem de parâmetros para uma função ou procedimento pode ser feita de duas formas:
 - **Por valor**
 - **Apenas o valor** é passado para a sub-rotina.
 - Após a execução da sub-rotina, a variável externa utilizada para passar o valor como parâmetro não tem o seu valor alterado.
 - **Por referência (uso de ponteiros)**
 - Neste caso, o endereço de memória da variável é passada à sub-rotina, e não apenas o seu valor.
 - Ao final da execução da sub-rotina, o valor do parâmetro **permanece** na variável original, uma vez que o valor foi atribuído ao endereço que referencia a variável.

Passagem de Parâmetros

- A passagem de parâmetros é feita através da especificação de “variáveis temporárias” entre os parênteses após o nome da rotina.

➤ Exemplo

```
void calculaArea (float largura, float comprimento){  
    ...  
}  
int calculaFatorial (int n){  
    ...  
}
```

- Não existe limites na passagem de parâmetros.



Passagem de Parâmetros

Procedimento **sem** parâmetros

```
1 #include<stdio.h>
2 // procedimento
3 void soma () {
4     int a, b, result;
5     printf("\nInforme dois valores: \n");
6     scanf("%d", &a);
7     scanf("%d", &b);
8     result = a + b;
9     printf("\nResultado: %d", result);
10 }
11
12 // rotina principal
13 main (void) {
14     printf("\nInicio da execucao - main");
15
16     soma();           // invocando o procedimento
17
18     printf("\nFim da execucao");
19     getch();
20 }
```

Procedimento **com** parâmetros

```
1 #include<stdio.h>
2
3 void soma(int x, int y) {
4     int result;
5     result = x + y;
6     printf("\nResultado: ", result);
7 }
8
9 main (void) {
10     int a, b;
11     printf("\nInforme dois valores: \n");
12     scanf("%d", &a);
13     scanf("%d", &b);
14
15     soma(a, b);
16
17     getch();
18 }
```

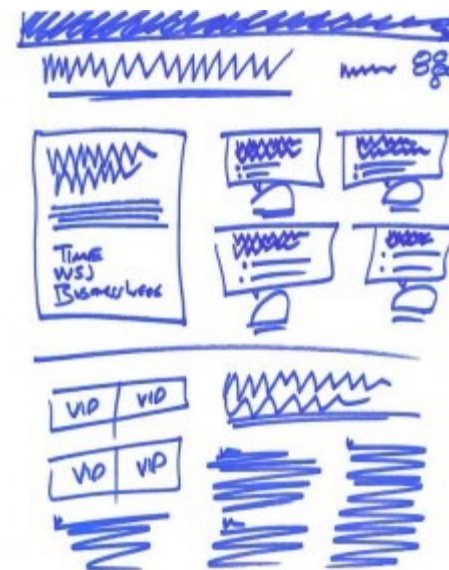


Exercício 03

- Faça um programa que contenha uma estrutura de caso que apresente as seguintes opções
 1. Maior nota (*// procedure*)
 2. Menor nota (*// procedure*)
 3. Média (*// function*)
 4. Necessário para ser aprovado no exame (*// function*)
- O programa deve receber duas notas informadas pelo usuário, que são passadas como parâmetro para as funções e procedimentos correspondentes às opções do menu. Os resultados devem ser apresentados na tela.



Protótipos



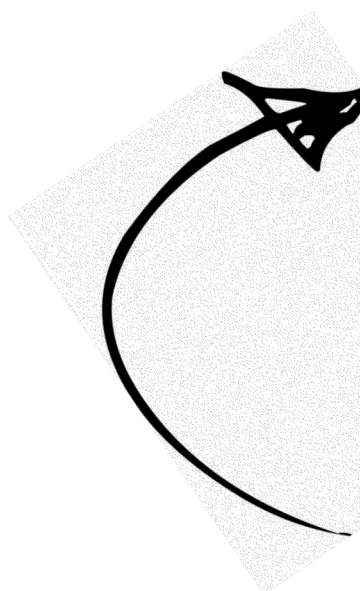


Protótipos

- Como a linguagem C é estruturada e sequencial, as funções e procedimentos devem ser escritos antes da rotina *main*.
- O uso de protótipos permite que as funções e procedimentos possam ser escritos após a *main*.
- Os protótipos funcionam como declaração de procedimentos/funções.
 - Indicam que a função/procedimento existe em algum lugar do código.

Protótipos

- Para adicionar o protótipo de uma função ou procedimento, basta adicionar a sua assinatura antes da rotina principal (*main*).
- Exemplo:



```

1 #include<stdio.h>
2 void soma(int x, int y);           // protótipo
3
4 main (void){                       // rotina principal
5     int a, b;
6     printf("\nInforme dois valores: \n");
7     scanf("%d", &a);
8     scanf("%d", &b);
9
10    soma(a, b);
11
12    getch();
13 }
14
15 void soma(int x, int y){           // procedimento
16     int result;
17     result = x + y;
18     printf("\nResultado: %d", result);
19 }
  
```



Exercício 04

- Organize os códigos-fonte dos exercícios anteriores de maneira que a rotina *main* fique no início do código, seguidos das funções e procedimentos devidamente comentadas.



Escopo de variáveis

Variáveis locais X Variáveis globais





Escopo de variáveis

- Com a utilização de sub-rotinas, surge a necessidade de diferenciar as variáveis.
 - **Variáveis Globais**
 - São declaradas no início do código, fora de rotinas e sub-rotinas.
 - Têm validade (podem ser acessadas) em qualquer parte do código (funções, procedimentos e *main*).
 - **Variáveis Locais**
 - São declaradas **dentro** de sub-rotinas.
 - Só podem ser acessadas pela sub-rotina onde foram declaradas.



Escopo de variáveis

- Exemplo

```
1 #include<stdio.h>
2 float pi = 3.14;           // variável global
3
4 float calculaVolume(float r, float a); // protótipo
5
6 main (void){               // rotina principal
7     float raio, altura;    // variáveis locais
8     printf("\nInforme dois valores: \n");
9     scanf("%f", &raio);
10    scanf("%f", &altura);
11
12    printf("Resultado: %f", calculaVolume (raio, altura));
13
14    getch();
15 }
16
17 float calculaVolume(float r, float a){ // procedimento
18     float result;                     // variável local
19     result = pi * (r * r) * a;
20     return result;
21 }
```



Exercício 05

- Faça um programa em C que contenha uma variável global inteira *A* inicializada com um valor qualquer, uma variável local inteira (também inicializada com um valor qualquer) na rotina *main*. Crie uma função teste para imprimir as duas variáveis.
- Comente o comportamento do programa.



Na próxima aula

- Manipulação de ponteiros



Referências

- SEBESTA, R. W. Conceitos de Linguagem de Programação. 4 ed. Bookman Companhia Ed. 2000.
- KERNIGHAM, Brian W.; RITCHIE, Dennis M. C: A Linguagem de Programação. Rio de Janeiro: Campus, 2002.
- DEITEL, H. M.; DEITEL, P. J. Como Programar em C. Rio de Janeiro: LTC - Livros Técnicos e Científicos, 1999.